

ZKOUŠKOVÉ PROCVIČOVÁNÍ - JAVA OPR1

Téma: Kurýrní doručovací služba

Obtížnost: vyšší střední - navazuje na dědičnost, polymorfismus a rozhraní, nově přidává jednoduchý návrhový vzor Strategy.

CÍL ÚKOLU

Vytvořte konzolovou aplikaci v Javě, která bude evidovat zásilky v kurýrní službě. Každá zásilka může být jiného typu a způsob výpočtu doručení se bude měnit podle zvolené doručovací strategie.

Úkol má procvičit hlavně:

- dědičnost a abstraktní třídu
- překrývání metod pomocí @Override
- polymorfismus při ukládání různých zásilek do jedné kolekce
- rozhraní a návrhový vzor Strategy
- práci s ArrayList a jednoduchým výpisem do konzole

POPIS SITUACE

Kurýrní firma přijímá různé typy zásilek: dopisy, balíky a potraviny. Každá zásilka má společné údaje, ale některé typy mají navíc vlastní atribut. Objednávka obsahuje více zásilek a má nastavený způsob doručení. Zákazník může například zvolit standardní doručení, expresní doručení nebo osobní odběr.

POŽADOVANÁ STRUKTURA TŘÍD

1. Abstraktní třída Zasilka

Vytvořte abstraktní třídu Zasilka s atributy:

- id - jedinečné označení zásilky
- prijemce - jméno příjemce
- mesto - město doručení
- hmotnostKg - hmotnost zásilky v kg
- zakladniCena - základní cena zásilky

Třída bude obsahovat konstruktor, gettery a metody:

```
public abstract String getTyp();
public String popis();
public double vypocitejZakladniCenu();
```

Metoda popis() vrátí text s ID, typem zásilky, příjemcem, městem a hmotností. Metoda vypocitejZakladniCenu() může například vracet zakladniCena + hmotnostKg * 10.

2. Podtřídy zásilky

Třída	Dědí z	Atribut navíc	Specifické chování
Dopis	Zasilka	boolean prioritni	pokud je prioritní, připočítá k ceně 30 Kč
Balik	Zasilka	boolean krehke	pokud je křehký, připočítá k ceně 50 Kč
Potraviny	Zasilka	boolean chlazene	pokud jsou chlazené, připočítá k ceně 80 Kč

Každá podtřída překryje metody `getTyp()`, `popis()` a podle potřeby i `vypocitejZakladniCenu()`.

NÁVRHOVÝ VZOR STRATEGY

V této úloze použijete návrhový vzor Strategy. Objednávka nebude sama pevně rozhodovat, jak se má doručovat. Místo toho bude mít uložený objekt strategie, který umí spočítat cenu doručení a vypsát způsob doručení. Díky tomu půjde způsob doručení změnit za běhu programu bez úprav třídy Objednavka.

3. Rozhraní DorucovacíStrategie

```
public interface DorucovacíStrategie {  
    String getNazev();  
    double spocitejCenuDoruceni(Zasilka zasilka);  
    String doruc(Zasilka zasilka);  
}
```

4. Konkrétní strategie doručení

Vytvořte tři třídy, které implementují rozhraní DorucovacíStrategie:

- StandardniDoruceni - cena doručení je 60 Kč + 5 Kč za každý kg, výpis např. „doručeno běžným kurýrem“
- ExpresniDoruceni - cena doručení je 120 Kč + 15 Kč za každý kg, výpis např. „doručeno expresně do druhého dne“
- OsobniOdber - cena doručení je 0 Kč, výpis např. „připraveno k vyzvednutí na pobočce“

5. Třída Objednavka

Vytvořte třídu Objednavka s atributy:

- cisloObjednavky - například String
- zasilky - kolekce ArrayList<Zasilka>
- strategie - objekt typu DorucovacíStrategie

Třída bude obsahovat metody:

```
public void pridejZasilku(Zasilka zasilka);  
public void setStrategie(DorucovacíStrategie strategie);  
public double spocitejCelkovouCenu();  
public void vypisObjednavku();
```

Celková cena objednávky se spočítá jako součet základní ceny každé zásilky a ceny doručení podle aktuálně nastavené strategie.

6. Třída Main

V metodě main proveďte následující:

- Vytvořte alespoň 5 různých zásilek - kombinujte dopisy, balíky a potraviny.
- Vytvořte jednu objednávku a přidejte do ní zásilky.
- Nastavte objednávce strategii StandardniDoruceni a vypište objednávku.
- Poté změňte strategii na ExpresniDoruceni a znovu vypište objednávku.
- Nakonec změňte strategii na OsobniOdber a znovu vypište objednávku.
- Při výpisu musí být vidět, že stejné zásilky mají různou celkovou cenu podle zvolené strategie.

UKÁZKA OČEKÁVANÉHO VÝSTUPU

```
Objednávka OBJ-2026-01  
Strategie: Standardní doručení  
-----  
Dopis D001 pro Jan Novák, Praha, 0.2 kg - prioritní  
Cena zásilky: 82.0 Kč, doručení: 61.0 Kč  
Balík B001 pro Eva Malá, Brno, 3.5 kg - křehké  
Cena zásilky: 185.0 Kč, doručení: 77.5 Kč  
Potraviny P001 pro Petr Svoboda, Ostrava, 4.0 kg - chlazené
```

Cena zásilky: 220.0 Kč, doručení: 80.0 Kč

Celkem: 705.5 Kč

Změna strategie na: Expresní doručení

Celkem: 902.5 Kč

Změna strategie na: Osobní odběr

Celkem: 487.0 Kč

Číselné hodnoty se nemusí přesně shodovat s ukázkou, pokud si zvolíte vlastní výpočet. Důležité je, aby byl výpočet konzistentní a aby změna strategie ovlivnila cenu doručení.

POVINNÉ POŽADAVKY

- Použijte alespoň jednu abstraktní třídu.
- Použijte alespoň jedno rozhraní.
- Použijte návrhový vzor Strategy.
- Použijte dědičnost pomocí extends.
- Použijte překrývání metod pomocí @Override.
- Použijte polymorfismus - například ArrayList<Zasilka>, ve kterém budou uloženy objekty tříd Dopis, Balík a Potraviny.
- Použijte konstruktory a volání super(...).
- Program musí jít spustit z metody main a musí vypisovat testovací data do konzole.

DOPORUČENÉ ROZDĚLENÍ SOUBORŮ

```
Zasilka.java
Dopis.java
Balik.java
Potraviny.java
DorucovacíStrategie.java
StandardniDoruceni.java
ExpresniDoruceni.java
OsobniOdber.java
Objednavka.java
Main.java
```

BONUSOVÉ ÚKOLY

- Ošetřete situaci, kdy objednávka nemá nastavenou žádnou strategii doručení.
- Přidejte metodu najdiZasilkuPodleId(String id), která vrátí nalezenou zásilku nebo null.
- Implementujte equals() a hashCode() u třídy Zasilka podle atributu id.
- Zabraňte vložení zásilky se stejným ID do jedné objednávky.
- Přidejte čtvrtou strategii, například VikendoveDoruceni, bez úpravy třídy Objednavka.

KRÁTKÁ NÁPOVĚDA K MYŠLENCE STRATEGY

Třída Objednavka by neměla obsahovat velký if/switch typu „pokud je doručení expresní, počítej takto; pokud je standardní, počítej jinak“. Místo toho pouze zavolá metodu na objektu strategie:

```
double cenaDoruceni = strategie.spocitejCenuDoruceni(zasilka);
```

Každá konkrétní strategie si sama rozhodne, jak cenu spočítá. To je hlavní princip tohoto návrhového vzoru.

KONTROLNÍ OTÁZKY PO DOKONČENÍ

- Kde se v řešení projevuje polymorfismus?
- Proč je vhodné použít rozhraní DorucovacíStrategie?
- Jak se změní program, pokud přidáte novou strategii doručení?
- Jaký je rozdíl mezi typem proměnné DorucovacíStrategie a konkrétním objektem ExpresniDoruceni?
- Proč by v tomto zadání nebylo ideální řešit doručení pomocí mnoha podmínek přímo ve třídě Objednavka?

Poznámka: Zadání je záměrně o něco těžší než předchozí úkol se střední obtížností, ale pořád drží rozsah běžného zkouškového příkladu. Nevyžaduje databázi, GUI, práci se soubory ani složité algoritmy.